

Access Modifiers & Encapsulation

1. The Three (or Four) Levels of Access

C++

```
class Car {
private:
    double price;           // accessible ONLY within this class (and its friends)
protected:
    string engineType;      // accessible within this class AND derived classes
public:
    string brand;           // accessible from anywhere
};
```

- **private** (default for **class**) — accessible only inside the class itself (and any explicitly-declared **friend**s — recall the previous topic).
- **protected** — accessible inside the class **and** any class that derives from it, but not from arbitrary outside code.
- **public** — accessible from anywhere the object itself is accessible.
- **Default access** differs by keyword: **class** defaults to **private**; **struct** defaults to **public** — this is, in fact, the **only** structural difference between **class** and **struct** in C++.

Java — one extra level: package-private

```
package com.kmotors.inventory;

public class Car {
    public String brand;           // accessible from ANYWHERE
    protected double baseCost;    // accessible within package + subclasses anywhere
    String internalNote;          // NO modifier = "default"/package-private
    private String internalCode;  // accessible ONLY within this class
}
```

The full Java access matrix — a very commonly tested table

Modifier	Same class	Same package	Subclass (different package)	Different package (non-subclass)
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
<i>(default/package-private)</i>	✓	✓	✗	✗
private	✓	✗	✗	✗

The row that catches people out: `protected` vs. `default`. Both allow same-package access identically — the *only* difference between them is that `protected` **additionally** allows access from a subclass **even in a different package**, while `default/package-private` does not extend that far. This is precisely why `protected` is described as "for inheritance" — it's the one modifier specifically designed to survive a subclass moving to a different package while still granting it access.

C++ has no direct equivalent of Java's `package-private` — C++'s three levels (`private`/`protected`/`public`) don't have a "same-file/same-directory" access tier; the closest structural analog for grouping related classes with shared internal access is C++'s `friend` mechanism (previous topic) or putting classes in an unnamed/detail namespace by convention.

2. Why Encapsulation (Data Hiding) Actually Matters

This was previewed in OOP Fundamentals — here's the fuller reasoning:

Protecting invariants

```
class BankAccount {
    double balance;
public:
    void withdraw(double amount) {
        if (amount > balance) throw runtime_error("Insufficient funds");
        balance -= amount;
    }
};
```

If `balance` were public, **any** code anywhere could set it directly (`account.balance = -500;`), bypassing the validation entirely. Because it's private, `withdraw()` is the **only** path that can ever change it — which means the invariant "balance never goes negative through a withdrawal" is actually **enforceable**, not just documented/hoped-for.

Enabling implementation changes without breaking callers

```
class Rectangle {
    private double width, height;
    public double getArea() { return width * height; } // currently: stored dimensions, computed area
}
// later, refactored internally:
class Rectangle {
    private double area; // now: stored area directly
    public double getArea() { return area; } // callers' code is COMPLETELY UNCHANGED
}
```

Because external code only ever called `getArea()` — never touched `width` / `height` directly — the entire internal representation could be swapped **without updating a single caller**. This is exactly the *why* behind "always use getters/setters instead of public fields," made concrete.

Read-only exposure (immutability via encapsulation)

```
class Point {
    private final int x, y;                // 'final' = cannot be reassigned after construction
    public Point(int x, int y) { this.x = x; this.y = y; }
    public int getX() { return x; }        // getter only – NO setX()
    public int getY() { return y; }
}
```

Providing **only** a getter (no setter) makes a field effectively **read-only from the outside** — combined with **final** (preventing even internal reassignment after construction), this is the standard Java pattern for building genuinely **immutable** objects, which are inherently thread-safe (no mutable shared state means no possibility of data races) and easier to reason about.

3. Static Members — class-level, not object-level

A **static** member belongs to the **class itself**, not to any individual object — there's exactly **one** copy, shared across every instance.

```
class Car {
public:
    static int totalCarsCreated; // declaration
    Car() { totalCarsCreated++; } // every constructor call increments the ONE shared counter
};
int Car::totalCarsCreated = 0;   // definition (required, outside the class, in C++)
```

```
class Car {
    static int totalCarsCreated = 0; // Java allows initialization directly here
    Car() { totalCarsCreated++; }
}
```

- **Static variable** — shared state across **all** instances (e.g., a running count of how many objects have been created).
- **Static method** — callable on the **class itself**, without needing any instance
(`Car.getTotalCars()` / `Car::getTotalCars()`) — and, correspondingly, a static method **cannot** access non-static (instance) members, since it doesn't run in the context of any particular object at all (there's no `this` / `*this` for it to use).

Interview soundbite: "Static is the mechanism for anything that's a property of the *class as a concept*, rather than of any specific object — a running total, a shared configuration constant, a utility/factory function that doesn't need any particular instance's state to do its job."

4. Packages (Java) — organizing and further controlling access

```
package com.kmotors.inventory; // MUST be the first line in the file

public class Car { /* ... */ }
```

- A **package** groups related classes, preventing name collisions (two unrelated `Car` classes can coexist in different packages) and forming the boundary that package-private and protected access rely on.
- **import** brings a class (or entire package) into scope so it can be referenced without full qualification:

```
java import com.kmotors.inventory.Car; // import a single class import
com.kmotors.inventory.*; // import every class in a package import static java.lang.Math.sqrt;
// static import – use sqrt() directly, unqualified
```

Without the import: `com.kmotors.inventory.Car nexon = new com.kmotors.inventory.Car();` — fully qualified, cumbersome but unambiguous. With it: `Car nexon = new Car();`

Interview Questions With Answers

Q1. What is the only structural difference between `class` and `struct` in C++?

Answer: Default member access. A `class`'s members default to `private` if no access specifier is given; a `struct`'s members default to `public`. Every other capability (methods, inheritance, constructors, etc.) is identical between the two — `struct` in C++ is not the limited, data-only construct it is in C; it's fully equivalent to `class` except for this one default.

Q2. What's the precise difference between Java's `protected` and default (package-private) access?

Answer: Both allow access from any class within the same package — they're identical in that respect. The difference is that `protected` additionally allows access from a subclass even if that subclass lives in a *different* package, while default/package-private access does not extend beyond the original package under any circumstance, subclass or not. This is exactly why `protected` is considered the modifier specifically designed to support inheritance across package boundaries.

Q3. Why does making a field private and only exposing it through a getter/setter allow the internal implementation to change without breaking calling code, using a concrete example?

Answer: If external code only ever interacts with an object through its public methods (like `getArea()`), it has no dependency on *how* that method computes its result internally — only that it returns the correct value. This means the internal representation can be completely restructured (e.g., switching from storing `width` / `height` and computing area on demand, to storing a precomputed

`area` field directly) without requiring any change to code that calls `getArea()`, since the public interface (the method's signature and behavior) stayed exactly the same even though the internals changed entirely.

Q4. How does encapsulation, combined with `final`, enable building immutable objects in Java?

Answer: Declaring a field `private` prevents any external code from directly modifying it at all — access is only possible through whatever methods the class chooses to expose. Providing only a getter (no setter) for that field means external code can read but never write it. Additionally marking the field `final` prevents even the class's *own* internal code from reassigning it after the constructor has run. Together, this guarantees the field's value is set exactly once at construction and can never change afterward, from any code, internal or external — which is precisely what "immutable" means, and is why immutable classes in Java are conventionally built with private final fields and getters only.

Q5. Why can't a static method access an instance (non-static) member directly?

Answer: A static method belongs to the class itself and can be called without any object existing at all (`Car.getTotalCars()`) — it has no associated `this` /current-instance context to operate on. An instance member, by definition, exists as part of a specific object's state — accessing it requires knowing *which* object's copy of that member is meant. Since a static method call carries no such object context, there's no instance for it to reference when trying to access a non-static member — the compiler rejects this because the reference would be fundamentally ambiguous/meaningless (which object's field would it even mean?).

Q6. Why is a static variable described as having "exactly one copy shared across every instance," and what's a good real-world use for this?

Answer: Unlike an instance variable (where every object gets its own independent copy, stored as part of that object), a static variable is stored once, associated with the class definition itself, rather than with any individual object — every instance of the class reads and writes that same single shared storage location. A good use case is a running count of how many objects have been created (e.g., `totalCarsCreated`), since that's inherently a fact about the class/type as a whole, not a property any single `Car` object individually owns — incrementing it in the constructor means every new object's creation is reflected in that one shared counter, visible identically from every instance or from the class itself.

Q7. Why does C++ have no direct equivalent to Java's package-private access level, and what's the closest structural analog?

Answer: C++'s access control model (private/protected/public) was designed around class and inheritance boundaries specifically, without an additional language-level grouping concept equivalent to Java's package system tied to directory/namespace structure and access rules. The closest structural analogs in C++ are the `friend` mechanism (explicitly grant specific external functions/classes access, as covered in the previous topic) or organizing related classes together and relying on

convention/documentation rather than a compiler-enforced "same group" access tier — neither is a perfect equivalent, since Java's package-private is enforced automatically based on physical package membership, while C++'s alternatives require explicit, per-relationship declarations.

Q8. Scenario: A team's `Employee` class in Java has a `protected double salary` field, intending for it to be accessible to subclasses like `Manager` and `Engineer` for calculating bonuses, but NOT accessible to unrelated classes. A new class `PayrollReport`, in a completely different package and NOT a subclass of `Employee`, is later found to be reading `salary` directly. How was this possible, and what does this reveal about a common misunderstanding of `protected` ?

Answer: This reveals a common misunderstanding: `protected` access in Java is not restricted to "only subclasses" in an absolute sense — it also grants access to any class within the *same package* as `Employee`, regardless of whether that class is a subclass at all. If `PayrollReport` were in the *same package* as `Employee`, it could access the protected `salary` field directly despite having no inheritance relationship to `Employee` whatsoever — this is the package-level component of `protected` access that's often overlooked (people tend to remember "protected = for subclasses" and forget "protected = for subclasses *and* the whole same package"). Given the description states `PayrollReport` is in a "completely different package" and not a subclass, protected access alone should *not* have permitted this — so the actual access must be happening through some other channel (e.g., `PayrollReport` might itself be a subclass despite the description, or there's a reflection-based access bypass, or the field was mistakenly declared `public` at some point) — the described scenario as stated, with genuinely different-package and non-subclass status, should be a compile error under `protected`, making this a good prompt for verifying the actual modifier and package/inheritance relationships rather than assuming the described constraints are accurate.